04_info_2604182225

Informatique
hors Python

En marge de l'apprentissage de Python 3, ce document traite de notions théoriques de l'informatique. Il se veut le complément du support dédié à la programmation, afin de couvrir l'ensemble de l'exigible.

Jean Decroocq

CPGE TSI – 2025–2027
Lycée polyvalent de Cachan

Dernière mise à jour : 18/04/2026
Communiquer une anomalie : jeandecroocq [at] hotmail.com

Table des matières

1 Intégration numérique **3**
1.1 Introduction 3
1.2 Méthode du point milieu 3
1.3 Méthode des trapèzes 4
1.4 Méthode de Simpson 5

2 Représentation des nombres en mémoire **6**
2.1 Bases de numération 6
2.2 Représentation des entiers 6
2.3 Représentation des nombres réels 7
2.4 Représentation des chaînes de caractères 8

1 Intégration numérique

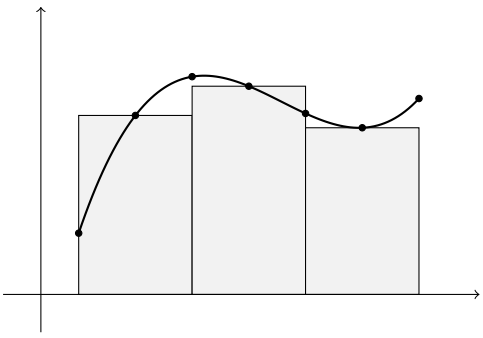
1.1 Introduction

- L'intégration numérique a pour objectif d'évaluer de façon approchée l'intégrale d'une fonction sur un intervalle borné $[a, b]$. Cette évaluation s'effectue à partir d'un ensemble fini de points, issus d'un calcul analytique ou de mesures physiques.
- La répartition des points sur l'axe des abscisses est généralement uniforme, définie par un pas d'échantillonnage constant noté h.
- Le principe fondamental repose sur l'interpolation polynomiale par morceaux. L'intervalle est découpé en sous-intervalles sur lesquels la fonction est approchée par un polynôme de degré faible (généralement 0, 1 ou 2), dont l'intégrale est aisée à calculer.
- La précision du calcul s'améliore en augmentant le nombre de points n (ce qui diminue le pas h) ou en élevant le degré du polynôme d'interpolation, sous réserve que la fonction étudiée présente de bonnes propriétés de continuité.

1.2 Méthode du point milieu

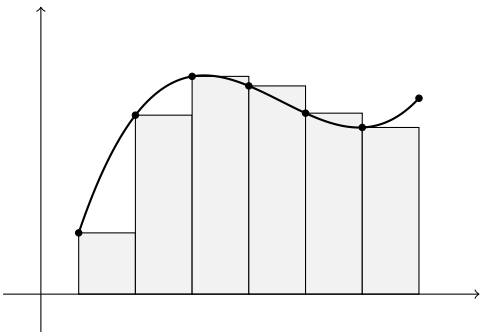
- La méthode du point milieu s'appuie sur une interpolation polynomiale de degré 0. La fonction est considérée comme constante sur chaque intervalle.
- Dans son approche théorique centrée sur des intervalles de largeur $2h$, la valeur de la fonction est évaluée au milieu de l'intervalle. L'intégrale s'écrit alors (pour un nombre de points n impair)

$$I = 2h \sum_{i=1}^{(n-1)/2} y_{2i}.$$



- Cette méthode ne requière en réalité qu'un échantillonnage de $2h$. L'intégration s'effectue alors de manière causale en utilisant la valeur du point courant sur une largeur h :

$$I = h \sum_{i=1}^{n-1} y_i.$$



Cette seconde méthode, bien que plus directe à implémenter, induit un décalage temporel systématique. L'aire calculée sur l'intervalle $[x_i, x_{i+1}]$ utilise la valeur y_i. Le résultat numérique est donc une approximation de l'intégrale d'une fonction déphasée, ce qui peut constituer une source d'erreur significative dans certaines applications.

– Afin d'implémenter cette méthode algorithmiquement, on propose la fonction suivante qui prend en argument une liste `Y` contenant les ordonnées successives des points, ainsi que le pas d'échantillonnage `h`.

```python
def integ_point_gauche(Y, h):
    s = 0
    for i in range(len(Y) - 1): # sans le dernier point
        s += h * Y[i]
    return s
```

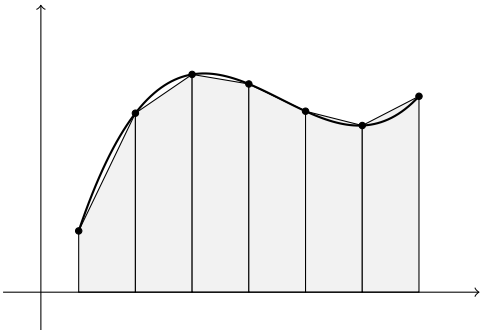
– Notons qu'on peut aussi employer le « point de droite ».

1.3 Méthode des trapèzes

– La méthode des trapèzes affine l'approximation en utilisant une interpolation linéaire (polynôme de degré 1) entre chaque couple de points consécutifs. L'aire sous la courbe est ainsi assimilée à une somme d'aires de trapèzes.

– En sommant les aires sur l'intervalle global, les ordonnées des points intérieurs sont comptabilisées deux fois, tandis que celles des extrémités ne le sont qu'une seule fois. La formule générale s'écrit donc

$$I = h \sum_{i=}^{n-1} \frac{y_i + y_{i+1}}{2}.$$



– Une implémentation de cette méthode en Python est la suivante.

```python
def integrer_trapezes(Y, h):
    s = 0
    for i in range(len(Y) - 1):
        s += (Y[i] + Y[i + 1]) / 2
    return h * s
```

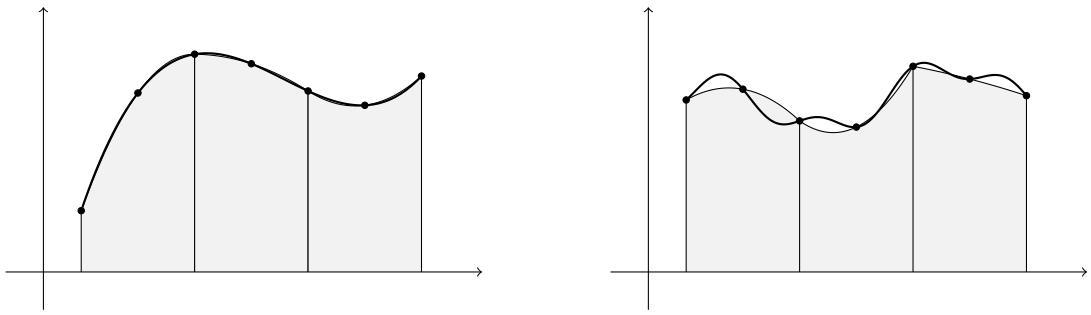
1.4 Méthode de Simpson

- La méthode de Simpson pousse la précision à un niveau supérieur en s'appuyant sur une interpolation quadratique, c'est-à-dire en approximant la fonction par un arc de parabole (polynôme de degré 2).
- Une parabole étant définie de manière unique par trois points, le calcul s'effectue par segments successifs de largeur $2h$, chacun délimité par un triplet de points $(y_{2i-1}, y_{2i}, y_{2i+1})$. Cette contrainte impose que le nombre total de points n soit impair.
- L'intégration analytique de la parabole unique passant par ces trois points aboutit à la formule locale, connue sous le nom de formule de Simpson

$$I_{local} = \frac{h}{3}(y_{2i-1} + 4y_{2i} + y_{2i+1}).$$

- En appliquant cette formule à chaque segment de trois points sur l'ensemble de l'intervalle, la valeur approchée globale de l'intégrale est donnée par la somme suivante

$$I = \frac{h}{3} \sum_{i=1}^{(n-1)/2} (y_{2i-1} + 4y_{2i} + y_{2i+1}).$$



- Bien que mathématiquement très précise pour les fonctions lisses, cette méthode est plus sensible aux bruits de mesure. Et dans un contexte temps réel, elle requiert la connaissance de trois points pour effectuer un calcul, induisant un retard temporel de $2h$.
- L'implémentation de cet algorithme parcourt la liste des ordonnées par pas de deux afin de traiter les points par triplets successifs.

```python
def integ_simpson(Y, h): # le nombre de points doit etre impair
    s = 0
    for i in range(0, len(Y) - 2, 2): # pas de 2
        s += Y[i] + 4 * Y[i+1] + Y[i+2]

    return s * (h / 3)
```

2 Représentation des nombres en mémoire

2.1 Bases de numération

- Le système décimal (base 10) est la norme de référence courante, mais l'architecture matérielle des ordinateurs repose sur le système binaire (base 2), dont les uniques symboles sont 0 et 1.
- L'écriture d'un nombre entier naturel dans une base B se décompose polynomialement. Un nombre représenté par la séquence $a_n a_{n-1} \dots a_1 a_0$ en base B possède une valeur décimale équivalente définie par
$$\sum_{k=0}^{n} a_k B^k,$$
où chaque chiffre a_k appartient à l'intervalle d'entiers $[0, B-1]$.
- Le système hexadécimal (base 16) est privilégié en électronique numérique car il permet une représentation très compacte des séquences binaires. Il utilise les chiffres de 0 à 9, complétés par les lettres de A à F (de 10 à 15).
- Le passage d'une base à une autre repose sur des méthodes algorithmiques :
  - D'une base B vers la base 10 : Évaluation du polynôme des puissances successives de la base. Par exemple, $BC5_{16} = 11 \times 16^2 + 12 \times 16^1 + 5 \times 16^0 = 3013_{10}$.
  - De la base 10 vers une base B : Divisions euclidiennes successives du quotient par la base B jusqu'à obtenir un quotient nul. La lecture des restes de bas en haut (du dernier au premier) donne l'écriture dans la nouvelle base.
  - Entre les bases 2 et 16 : Une conversion directe est possible en regroupant les bits par paquets de quatre (un quartet ou *nibble*). Par exemple, la lecture du motif $1100\,0000_2$ s'associe directement à $C0_{16}$.
- Le langage Python gère nativement ces différentes bases au moyen de préfixes et de fonctions de conversion spécifiques.

```python
# saisie directe via prefixe
valeur_bin = 0b1101  # base 2 (vaut 13)
valeur_hex = 0xA3    # base 16 (vaut 163)

# conversion d'un entier vers une chaine
chaine_bin = bin(13)  # renvoie '0b1101'
chaine_hex = hex(163) # renvoie '0xa3'
```

2.2 Représentation des entiers

- Les opérations au sein du microprocesseur s'effectuent sur des blocs d'information appelés *mots-machine*, dont la taille n s'exprime en bits (généralement 32 ou 64 bits). L'*octet* (8 bits) constitue l'unité usuelle de stockage en mémoire.
- Les entiers naturels sont encodés en binaire direct. Un mot de n bits permet de couvrir l'intervalle d'entiers allant de 0 à $2^n - 1$. Le dépassement de cette borne supérieure engendre une erreur matérielle par dépassement de capacité (*overflow*). Python, en revanche, gère dynamiquement la taille des entiers en s'affranchissant de cette limite matérielle.
- Le codage des entiers relatifs (signés) s'effectue par la méthode du *complément à 2*. L'intervalle de représentation est translaté pour s'étendre de -2^{n-1} à $2^{n-1} - 1$.

- Dans cette représentation, le bit de poids fort indique le signe de la valeur : 0 désigne un entier positif, et 1 un entier négatif.
- L'obtention du code d'un entier négatif a (où $a < 0$) sur n bits peut se faire par deux approches :
  - Approche mathématique : Coder en binaire direct la valeur positive résultant de l'opération $a + 2^n$.
  - Approche algorithmique : Écrire la valeur absolue de a en binaire, inverser tous les bits (les 0 deviennent 1, et inversement), puis ajouter la valeur 1 au résultat.

2.3 Représentation des nombres réels

- La conversion d'un nombre à virgule en binaire requiert un traitement séparé de sa partie entière et de sa partie fractionnaire.
- La méthode de conversion de la partie fractionnaire consiste à la multiplier successivement par 2. À chaque itération, la partie entière du résultat (nécessairement 0 ou 1) constitue le chiffre binaire suivant de la partie fractionnaire, de gauche à droite. Le processus se répète avec la nouvelle partie fractionnaire jusqu'à ce que celle-ci devienne nulle ou que la limite de la mémoire soit atteinte.
- Les ordinateurs utilisent la représentation en *virgule flottante* normalisée par l'IEEE 754. Cette norme attribue trois rôles distincts aux bits du mot-machine :
  - Le *signe* (s) : 1 bit (0 pour positif, 1 pour négatif).
  - L'*exposant* (e) : Décalé d'une valeur fixe (127 en simple précision sur 8 bits, 1023 en double précision sur 11 bits) pour éviter l'usage d'un exposant signé.
  - La *mantisse* (m) : Représente les chiffres significatifs de la partie fractionnaire.
- La conversion manuelle d'un réel (par exemple $-243,25$) vers la norme IEEE 754 s'opère ainsi :
  - Conversion en binaire complet : $-11110011,01_2$.
  - Décalage de la virgule (normalisation) : $-1,111001101_2 \times 2^7$. L'unique bit 1 avant la virgule est implicite et ne sera pas mémorisé.
  - Calcul de l'exposant décalé : $7 + 127 = 134 = 10000110_2$.
  - La mantisse est complétée par des zéros à droite pour atteindre la taille requise.
- L'évaluation de la valeur x à partir de ses champs (s, e, m) dépend du contenu de l'exposant :
  - *Nombres normalisés* : Si l'exposant n'est ni rempli de zéros ni de uns, la valeur est $x = (-1)^s \times (1, m_1 m_2 \dots) \times 2^{e-\text{décalage}}$.
  - *Nombres dénormalisés* : Si $e = 0$ et $m \neq 0$, le bit implicite devient 0, permettant de coder des nombres extrêmement proches de zéro.
  - *Zéro* : Si $e = 0$ et $m = 0$. Il existe ainsi un $+0$ et un -0.
  - *Infinis* : Si e est à sa valeur maximale (que des 1) et $m = 0$, le nombre représente $\pm\infty$.
  - *NaN (Not a Number)* : Si e est à sa valeur maximale et $m \neq 0$, signale une erreur ou une indétermination mathématique.

```python
# affichage de l'ecriture hexadecimale interne du flottant
valeur = 5.5
print(valeur.hex())
# affiche 0x1.6000000000000p+2
```

- La représentation fractionnaire binaire induit un inconvénient majeur : certains nombres décimaux simples, tels que $0,1$, possèdent un développement binaire infini et périodique. Ils ne peuvent donc pas être stockés de manière exacte et subissent une troncature matérielle.

- Ces approximations s'accumulent lors des opérations arithmétiques, entraînant des erreurs de précision notables. Par exemple, l'évaluation de `0.1 + 0.2` en Python renvoie `0.30000000000000004`.
- Il est par conséquent impératif de ne jamais vérifier l'égalité stricte entre deux nombres flottants avec l'opérateur `==`. La comparaison s'effectue en validant que la valeur absolue de leur différence demeure inférieure à une tolérance d'erreur infinitésimale.

2.4 Représentation des chaînes de caractères

- Les chaînes de caractères obéissent à une logique de représentation totalement différente. Elles nécessitent une table de correspondance liant un symbole à une séquence de bits.
- L'encodage historique ASCII associe 128 caractères à des codes sur 7 bits (stockés sur un octet). Pour pallier ses limites et intégrer l'intégralité des symboles internationaux, la norme moderne UTF-8 est utilisée, encodant les caractères sur une longueur variable d'octets.